

Chap 1

Date

No.

names

binary (base 2) octal (base 8) decimal (base 10) hexadecimal (base 16)

hexadecimal digits: 0, 1, ..., 9, A, B, ..., F

Conversion

1. Base-R to Decimal Conversion: multiple digits with their corresponding weights.

$$\text{eg. } (2A.8)_{16} = 2 \times 16^1 + 10 \times 16^0 + 8 \times 16^{-1} = (42.5)_{10}$$

2. Decimal to Base-R Conversion

① shortcut: if $R=2$ $(9)_{10} = 2^3 + 2^1 = (1010)_2$ sum-of-weights method

② repeated division-by-R method: use successive division by R until the quotient is 0

(whole numbers) bottoms-up

$$(43)_{10} = (101011)_2$$

2	43	
2	21	...1
2	10	...1
2	5	...0
2	2	...1
2	1	...0
	0	...1

↑ LSB
↓ MSB: most significant bit

mathematical basis:

Any integer N can be expressed in base R as: $0 \leq a_i < R$

$$N = a_0 + a_1R + a_2R^2 + \dots + a_kR^k, \text{ where } a_0, a_1, \dots, a_k \text{ are the digits in base } R$$

connection to the remainders:

- The remainder of N divided by R is exactly a_0 , the LSB
- The quotient $\lfloor N/R \rfloor$ equals $a_1 + a_2R + \dots + a_kR^{k-1}$, the remaining digits in base R .
- Repeating division extracts digits one by one from LSB to MSB
- Reverse order

③ repeated multiplication-by-R method: repeated multiplication by R is used, until the fractional product is 0 (or until the desired number of decimal places is achieved)

(fractions) top-down

$$(0.3125)_{10} = (0.0101)_2$$

$0.3125 \times 2 = 0.625$	↑ MSB
$0.625 \times 2 = 1.25$	
$0.25 \times 2 = 0.50$	
$0.5 \times 2 = 1.00$	↓ LSB

mathematical basis:

Let's say the fraction part is F . $\rightarrow$ where a_1 is the integer part

When you multiply F by R , $R \times F = a_1 + f_1$, f_1 is the new fraction part.

$$\text{This means: } F = a_1R^{-1} + f_1R^{-1}$$

second multiplication, $R \times f_1 = a_2 + f_2$

Besform

substitute back: $F = a_1 R^{-1} + a_2 R^{-2} + a_3 R^{-3}$

repeat indefinitely: $F = a_1 R^{-1} + a_2 R^{-2} + a_3 R^{-3} + \dots$

each a_i is the integer part obtained from multiplication - it becomes the i -th digit after the radix point in base R .

When at some step the new fractional part $f_n = 0$, it means all digits have been extracted and the conversion process stops.

★ each digit represents how many times the current place value (in base R) can fit into the remaining fractional part

⊕ binary-octal/hexadecimal conversion

- $B \rightarrow O$ partition in groups of 3 $(1011011001.101110)_2 = (2731.56)_8$
- $O \rightarrow B$ reverse

Binary Arithmetic Operations

1. $+$: carry propagation

2. $-$: $(11001)_2$

$-(10011)_2$

$(00110)_2$

Can't be omitted

3. $\times$

alignment

$(11001)_2$

$\times (10101)_2$

$(11001)_2$

$(11001)_2$

$+(11001)_2$

$(1000000101)_2$

4. $\div$

$(10001) \div (11)_2$

enough

$$\begin{array}{r} 101 \\ 11 \overline{) 10001} \\ \underline{01100} \\ 101 \\ \underline{11} \\ 10 \end{array}$$

Negative numbers

n bits $\rightarrow$ # of representations is 2^n

1. Sign-and-magnitude

Sign bit: 0 for +, 1 for -

eg. 8 bits num } 1-bit sign

$-(12)_{10} \sim +(12)_{10} = 255$

7-bits magnitude

(compared to n -bit unsigned binary number represents values from 0 to $2^n - 1$)

2. 1's complement: $2^n - x - 1$, where $0 \leq x \leq 2^{n-1} - 1$

eg. $-(00001100)_2 = (2^8 - 12 - 1)_{10} = (243)_{10} = (11110011)_2$

most negative: $2^8 - (2^7 - 1) - 1 = 128(10) = (10000000)_2$ requires a 9 bit x

shortcut: $1 \rightarrow 0, 0 \rightarrow 1$

3 2's complement: $2^n - x$ where $1 \leq x \leq 2^{n-1}$

shortcut: invert the original code of its absolute value + 1

" $x + (2^n - x)$ " = $2^n \rightarrow$ overflow, so the result is 0

That is: " $2^n - x$ " is equiv. to $-x$.

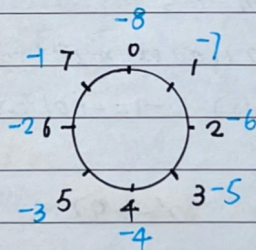
eg. 8 bits: $-128 \sim +127$

note that "100 000 00" is the only one that can't be derived from shortcut.

sign magnitude Δ 1111111 -1: the biggest among negative numbers

III Comparisons of Sign-and-Magnitude & Complements (4-bit signed number)

Value	S&M	1's	2's	Value	S&M	1's	2's
+7	0111	0111	0111	-0	1000	1111	—
+6	0110	0110	0110	-1	1001	1110	1111
+5	0101	0101	0101	-2	1010	1101	1110
+4	0100	0100	0100	-3	1011	1100	1101
+3	0011	0011	0011	-4	1100	1011	1100
+2	0010	0010	0010	-5	1101	1010	1011
+1	0001	0001	0001	-6	1110	1001	1010
+0	0000	0000	0000	-7	1111	1000	1001
				-8	—	—	1000



Adding 1 to 0111 (+7) gives 1000 (-8)

1 1111 (-1) 0000 (0)

Overflow problems.

in a computer, the operations of signed integers are performed using 2's

eg. $(1001)_2 + (1101)_2 = (10110)_2$ (discard end-carry)

$= (0110)_2$

$(-7)_{10} + (-3)_{10} = (6)_{10}$

Floating Point Numbers: is used to present a very small subset of real num. The num. of floating numbers is finite & countable.

eg. $0.23 \times 10^3, -0.1239 \times 10^{-10}$

• Floating point numbers have 3 parts: $0.52874 \times 10^{+2}$ — exponent

• Mantissa is usually in normalized form $0.17 \times 10^{+9}$ — mantissa base

(e.g. 0.17×10^2 , where $0.17 > 0$, in this case)

eg. -0.0017×10^{21} normalized to -0.17×10^{19}

normalization

$$0.01101(2) \times 2^3 = 0.1102(2) \times 2^2 \text{ (x2 shift left by one bit)}$$

M & E are represented by the 2's complement

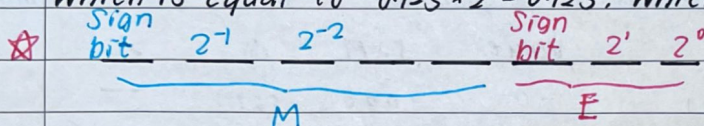
$$5m + 3e \rightarrow 8 \text{ bits.}$$

★ What will happen if the mantissa start with 00 & 11?

Short-coming: the floating-point number may not be close to the actual real ^{number}

eg. (4m+4e)

Without the normalization, the floating-point representation will be 000/0000, which is equal to $0.125 \times 2^0 = 0.125$, which will deviate quite far from 0.21875.



eg. 10011011 → 3

$$\hookrightarrow 1.0011 \Rightarrow 0.1101 = -(0.5 + 0.25 + 0.0625) = -0.8125(10)$$

① If the mantissa is positive, $0.5 \leq f < 1$

ie. 01000 (0.5) to 01111 ($0.5 + 0.25 + 0.125 + 0.0625 < 1$)

②

negative, $-1 \leq f < -0.5$

ie. 10000 (-1) to 10111 ($0.25 + 0.125 + 0.0625 - 1 < -0.5$)

eg. Convert -0.377(10) to notation with 5m & 3e bits

not smaller than -0.5

$$-0.377(10) = -0.754(10) \times 2^{-1} = -0.1100(2) \times 2^{-1}$$

$$\therefore 01100(2) = 10100(2's) \quad -1 = -001(2) = 111(2's) \Rightarrow 1010011$$

ASCII (7 bits)

'0' → 48

'A' → 65

'a' → 97

' ' → 32

'\0' → 0

'\n' → 10

int

4 bytes 1 byte = 8 bits

Identifier

- ① letters, digits, ...
- ② x begin with digits

Remainder

the remainder follows the sign of numerator, and the magnitude is equal to $\text{abs}(\text{number}) \% \text{abs}(\text{denom})$

Fields widths, justification & precision

printf("%-10s %7.2f\n", "Paper", 5.99);

left-justified width precision

① %8s ② %.8s

printf

Date

No.

- `%8s` means print the string in a field of at least 8 characters
- if the string is shorter than 8, spaces are added on the left (right-aligned)
- if the string is longer than 8, the full string is printed (no truncation)
- `%.8s` means print at most 8 characters of the string
- if the string is longer than 8, it's truncated
- if the string is shorter than 8, the whole string is printed without padding.

scanf

- `%ns` read at most n characters of a string
- ① stops when a whitespace is encountered (space, newline, tab)
- ② stops when n characters have been read